

PROYECTO N°1
Diseño de un Agente Inteligente usando Búsqueda Adversarial
Aplicación en un juego

Fecha de entrega 1: Domingo 23 de agosto de 2026

Fecha de evaluación 1: Semana 4 (del 24-agosto al 28-agosto)

Fecha de entrega 2: Domingo 06 de septiembre de 2026

Fecha de evaluación 2: Semana 6 (del 07-septiembre al 11-septiembre)

Modalidad: Trabajo Grupal (3 integrantes. No puede ser individual)

Ponderación: 20% de la nota del curso

1. Contexto.

La búsqueda adversarial es una de las aplicaciones de la Inteligencia Artificial para la toma de decisiones en entornos competitivos, donde dos agentes poseen objetivos opuestos y las decisiones de uno afectan directamente las posibilidades de éxito del otro. Este tipo de problemas se presenta en numerosos dominios, siendo los juegos de tablero de dos jugadores uno de los escenarios clásicos utilizados para el estudio y desarrollo de agentes inteligentes.

En este proyecto, cada grupo desarrollará un agente inteligente capaz de participar en un juego de tablero de dos jugadores mediante técnicas de búsqueda adversarial. Para ello, deberán modelar adecuadamente el problema, representar los estados del juego, generar los movimientos legales, identificar estados terminales y desarrollar un algoritmo que permita seleccionar la mejor acción posible considerando las respuestas potenciales del adversario.

El desarrollo del proyecto se realizará en dos etapas. En la primera, se implementará completamente el entorno del juego, incluyendo sus reglas y mecánicas. En la segunda, dicho entorno será utilizado para incorporar un **agente inteligente** basado en el algoritmo **Minimax**, optimizado mediante poda **Alfa-Beta** y apoyado por una **función de evaluación heurística** diseñada por el propio grupo.

El propósito de este proyecto no es desarrollar un videojuego ni obtener necesariamente el agente que siempre gane una partida. El foco principal consiste en aplicar correctamente los conceptos de búsqueda adversarial estudiados en el curso, diseñando un agente capaz de tomar decisiones fundamentadas dentro de las limitaciones computacionales propias de este tipo de problemas.

2. Objetivo del proyecto.

Desarrollar un **agente inteligente** para un juego de tablero de dos jugadores, aplicando técnicas de **búsqueda adversarial** mediante la implementación del algoritmo **Minimax**, optimizado con **poda Alfa-Beta** y apoyado por una **función de evaluación heurística**, integrando los

conocimientos adquiridos sobre modelamiento de problemas, representación de estados y toma de decisiones en entornos competitivos.

3. Selección del juego.

Cada grupo desarrollará su proyecto utilizando **uno de los siguientes juegos**:

- Gomoku
- Isolation
- Othello (Reversi)
- Breakthrough
- Domineering
- Ataxx
- Konane
- Dodgem
- Clobber
- Teeko

El(la) profesor(a) de taller será quien asigne un juego a cada grupo.

Todos los juegos deberán implementarse utilizando un tamaño de tablero parametrizable (como entrada en la ejecución) y será durante las presentaciones 1 y 2 del proyecto, cuando el(la) profesor(a) indicará con qué parámetros realice la prueba del código.

3.1 Restricciones para el tamaño del tablero.

Debido a las características propias de cada juego, deberán respetarse las siguientes restricciones:

Juego	Restricciones del tablero
Gomoku	Tablero de tamaño $n \times n$, con $n \geq 5$. La condición de victoria se mantiene en 5 fichas consecutivas .
Isolation	Tablero de tamaño $n \times n$, con $n \geq 4$. No existen restricciones adicionales sobre el tamaño del tablero.
Othello (Reversi)	Tablero de tamaño $n \times n$, donde n debe ser un número par mayor o igual a 4 . La disposición inicial de las cuatro fichas centrales deberá adaptarse automáticamente al tamaño del tablero.
Breakthrough	Tablero de tamaño $n \times n$, con $n \geq 6$. La configuración inicial de las fichas deberá adaptarse automáticamente al tamaño del tablero, manteniendo dos filas de piezas por jugador.
Domineering	Tablero de tamaño $n \times n$, con $n \geq 4$. No existen restricciones adicionales sobre el tamaño del tablero.
Ataxx	Tablero de tamaño $n \times n$, con $n \geq 5$. No existen restricciones adicionales sobre el tamaño del tablero.
Konane	Tablero de tamaño $n \times n$, con n par y $n \geq 6$. No existen restricciones adicionales sobre el tamaño del tablero.
Dodgem	Tablero de tamaño $n \times n$, con n par y $n \geq 3$. No existen restricciones adicionales sobre el tamaño del tablero.

Clobber	Tablero de tamaño $n \times n$, con n par y $n \geq 4$. No existen restricciones adicionales sobre el tamaño del tablero.
Teeko	Tablero de tamaño $n \times n$, con $n \geq 5$. No existen restricciones adicionales sobre el tamaño del tablero.

4. Sobre la implementación en Python.

- El proyecto deberá desarrollarse íntegramente en **Python**. No se permitirá el uso de bibliotecas externas **que tengan incluido el juego asignado**.
- La solución deberá presentar un **diseño modular**, organizado mediante funciones y/o clases con objetivos claramente definidos. Se espera una adecuada separación entre la lógica del juego, la interacción con el usuario y el agente inteligente **(PEP-8)**.
- Como referencia para la organización del código, los grupos deberán guiarse por el **ejemplo del código fuente del "Juego del Gato"** desarrollado en cátedra. Dicho ejemplo representa el nivel de modularidad, organización y calidad de programación esperado para este proyecto.
- El programa deberá implementar íntegramente la lógica del juego, incluyendo la representación del tablero, generación y aplicación de movimientos, control de turnos, detección de estados terminales, implementación de **Minimax, poda Alfa-Beta** y la **función de evaluación heurística**.
- El tamaño del tablero y los parámetros del algoritmo deberán ser configurables y no podrán estar codificados para un único caso particular.
- El código deberá ser claro, legible y correctamente comentado, utilizando identificadores descriptivos y evitando la duplicación de código y código basura **(PEP-8)**.

5. Sobre las entregas.

- Entrega 1: código totalmente operativo del juego asignado, implementado para 2 jugadores humanos.
- El grupo debe subir a la casilla de Canvas (Taller) la solución en un archivo **.zip** con todo el código en fuentes **.py**.
- Entrega 2: Código totalmente operativo del juego asignado, con el agente implementado.
- **No se aceptarán entregas atrasadas.**

6. Sobre la evaluación.

La evaluación del proyecto considerará tanto la **calidad técnica de la solución desarrollada** como la **capacidad de cada integrante del grupo para explicar, justificar y defender las decisiones de diseño adoptadas**.

Los aspectos específicos que serán considerados en cada entrega son los siguientes:

6.1 Entrega 1: Implementación del juego para 2 jugadores humanos (30%)

- Será en el horario de taller por lo que **todos los integrantes del grupo deberán estar presentes**.
- La evaluación de cada grupo se construirá considerando el resultado de la **interrogación** que se realizará. Aquí el profesor podrá:
 - Solicitar una o más demostraciones del comportamiento del juego entregado, **variando los parámetros de entrada**.
 - Formular preguntas dirigidas a cada integrante del grupo, sobre **cualquier aspecto del juego y su implementación**.
 - Solicitar cambios en la lógica del juego dirigidos a cada integrante del grupo, y el **integrante designado tendrá que hacerlo durante la interrogación**.

6.2 Entrega 2: Implementación del juego con el agente incorporado (70%)

- Será en el horario de taller por lo que todos los integrantes del grupo deberán estar presentes.
- La evaluación se construirá considerando dos etapas:
 - La **revisión del código** considerando: funcionalidad, robustez, modularidad, aplicación de buenas prácticas de programación. Esto valdrá un **20%** de la nota del proyecto.
 - La **interrogación** a cada grupo mostrando la ejecución del juego. Esto valdrá un **50%** de la nota del proyecto. Aquí el profesor podrá:
 - Solicitar una o más demostraciones del comportamiento del agente en el juego entregado.
 - Formular preguntas dirigidas a cada integrante del grupo, sobre **cualquier aspecto del agente y su implementación**.
 - Solicitar cambios en la **lógica del agente** dirigidos a cada integrante del grupo, y el **integrante designado tendrá que hacerlo durante la interrogación**.